

LibeRation reference manual

Population Pharmacometric Simulation and Estimation

Version 0.8.0

Generated 2026-07-22

100 help topics

Contents

- `liber_doctor`
- `liber_gui`
- `liber_workbench`
- `liberWorkbenchOutput`
- `nm_arma_config`
- `nm_benchmark_gate`
- `nm_benchmark_provenance`
- `nm_bootstrap`
- `nm_compile`
- `nm_component`
- `nm_component_code`
- `nm_control_read`
- `nm_control_write`
- `nm_cov_step`
- `nm_cthmm_config`
- `nm_dae_config`
- `nm_dataset`
- `nm_dde_config`
- `nm_diagram_generate`
- `nm_diagram_preview`
- `nm_est`
- `nm_est_sequence`
- `nm_est_stage`
- `nm_etab`
- `nm_experimental_config`
- `nm_factorial_chain`
- `nm_factorial_hmm_config`
- `nm_gof`
- `nm_hmm_config`
- `nm_hmm_decode`
- `nm_hsmm_config`
- `nm_individual_fit`
- `nm_irt_outcomes`
- `nm_kalman_config`
- `nm_kalman_decode`
- `nm_lik_config`
- `nm_matrix_model`
- `nm_mixture`
- `nm_mixture_posterior`
- `nm_model`
- `nm_model_contract_fields`
- `nm_model_cpp`
- `nm_model_diagram`
- `nm_model_diagram_get`
- `nm_model_outputs`
- `nm_model_template`
- `nm_npc`
- `nm_npde`
- `nm_objective`
- `nm_outcome`
- `nm_outcome_diagnostics`
- `nm_outcomes`
- `nm_piecewise`
- `nm_prediction_derivatives`
- `nm_prior`
- `nm_profile`
- `nm_project_copy`
- `nm_project_create`
- `nm_project_delete`
- `nm_project_delete_snapshot`
- `nm_project_list`
- `nm_project_load`
- `nm_project_load_diagnostics`
- `nm_project_save`
- `nm_project_save_diagnostics`
- `nm_project_save_run`
- `nm_qsp_model`
- `nm_qsp_system`
- `nm_re_block`
- `nm_re_config`

- nm_report
- nm_report_block
- nm_report_design
- nm_report_design_load
- nm_report_design_render
- nm_report_design_save
- nm_residual_group
- nm_scm
- nm_sde_config
- nm_simulate
- nm_spline
- nm_structural_templates
- nm_support_matrix
- nm_switching_state_space_config
- nm_validation_gate

- nm_vpc
- nm_vpc_categorical
- nm_vpc_competing
- nm_vpc_count
- nm_vpc_recurrent
- nm_vpc_tte
- nm_workspace
- nm_workspace_backup
- nm_workspace_gc
- nm_workspace_migrate
- nm_workspace_verify
- NMEngine
- predict.nm_fit
- renderLiberWorkbench
- residuals.nm_fit

Diagnose a LibeR ecosystem installation

Description

Reports package compatibility, compiled dependency provenance, wire-contract versions, workspace health, and common deployment mistakes. It does not modify the installation or workspace.

Usage

```
liber_doctor(workspace = NULL, strict = FALSE, verbose = interactive())
```

Arguments

workspace	Optional [nm_workspace()] or workspace directory to inspect.
strict	Throw an error if a required component is missing or incompatible.
verbose	Print the report.

Value

A 'liber_diagnostics' list.

Launch the LibeR React modelling application

Description

This is the interactive workbench used to manage model versions, datasets, simulations, estimations, diagnostics, reports, and local or remote jobs.

Usage

```
liber_gui(  
  model = NULL,  
  data = NULL,  
  queue = NULL,  
  workspace = NULL,  
  project = NULL,  
  session_workspace = FALSE,  
  launch.browser = getOption("shiny.launch.browser", interactive()),  
  ...  
)
```

Arguments

model	Optional model loaded into the workbench.
data	Optional dataset loaded into the workbench.
queue	Optional 'LibeRQueue'; simulation and estimation jobs are submitted to it. 'NULL' creates a persistent local queue under the workspace when LibeRties is installed. Use 'FALSE' for in-process work.
workspace	Workspace object or directory. By default the GUI uses 'C:/Users/<username>/Documents/LibeR/workspace' on Windows and '~/LibeR/workspace' on Linux and macOS.
project	Optional project id to open when the application starts.

`session_workspace`

Create a separate ephemeral workspace beneath 'workspace' for every browser session. This is intended for hosted demonstrations where application users must not share project files.

`launch.browser`

Passed to `[shiny::runApp()]`. Use 'NULL' to return the Shiny app object without launching it.

...

Additional arguments passed to `[shiny::runApp()]`.

Examples

```
if (interactive()) liber_gui()
```

React workbench for LibeRation

Description

React workbench for LibeRation

Usage

```
liber_workbench(  
  model = NULL,  
  data = NULL,  
  jobs = NULL,  
  result = NULL,  
  fit = NULL,  
  report = NULL,  
  report_design = NULL,  
  diagnostics = NULL,  
  hmm = NULL,  
  kalman = NULL,  
  log = NULL,  
  job_log = NULL,  
  server = NULL,  
  workspace = NULL,  
  library = NULL,  
  ai = NULL,  
  ai_context = NULL,  
  report_ai_context = NULL,  
  report_directory = "",  
  output_catalog = NULL,  
  run_output = if (inherits(fit, "nm_fit")) fit$output else NULL,  
  data_payload = TRUE,  
  gof_payload = TRUE,  
  diagnostic_payload = names(diagnostics),  
  input_id = "liber_workbench",  
  width = NULL,  
  height = "780px",  
  elementId = NULL  
)
```

Arguments

model	Optional [nm_model()] or [NMEngine].
data	Optional NONMEM-style dataset.
jobs	Optional job table, such as 'LibeRQueue\$list()'.
result	Optional latest simulation/result summary.
fit	Optional fitted model used for parameter and diagnostic panels.
report	Optional generated [nm_report()].
report_design	Optional saved [nm_report_design()] restored by the visual report designer.
diagnostics	Optional saved VPC, NPC, and NPDE results for the selected estimation run.
hmm	Optional decoded HMM GUI payload. When omitted, availability is inferred from the selected model and fit and rows remain lazy.
kalman	Optional decoded linear state-space GUI payload. When omitted, availability is inferred from the selected model and fit and rows remain lazy.
log	Optional application-log payload.
job_log	Optional stdout/stderr lines for the selected queued job.

server	Optional runtime/server status passed to the workbench.
workspace	Optional [nm_workspace()] and current project metadata.
library	Optional LibeRary catalogue payload.
ai	Optional browser-local WebGPU AI settings, independent Help and Report context-window choices, and model catalogue.
ai_context	Optional compact, on-demand summaries of saved project runs supplied to the browser-local Help model. Full result datasets are never included in this payload.
report_ai_context	Optional detailed, on-demand summaries of the runs selected in the visual report workflow.
report_directory	Default report output directory displayed by the visual report builder.
output_catalog	Optional draft output catalogue used by the GUI after validation but before editor changes are applied.
run_output	Optional selected model-run output columns, aligned by a '.ROW' column or by row position. These are loaded into Data explorer only when its row payload is requested.
data_payload	Whether dataset rows are included in the browser payload. When false, only dataset metadata and column names are included.
gof_payload	Whether GOF rows are generated when 'fit' is an [nm_fit].

`diagnostic_payload`

Names of saved diagnostics whose plot data should be included.
Availability metadata is always included.

`input_id`

Shiny event input prefix.

`width, height`

Widget dimensions.

`elementId`

Optional HTML widget element id.

Shiny output for the LibeR workbench

Description

Shiny output for the LibeR workbench

Usage

```
liberWorkbenchOutput(outputId, width = "100%", height = "100vh")
```

Arguments

outputId	Shiny output id.
width, height	CSS dimensions.

Configure an ARMA residual process

Description

Generates an exact linear state-space representation for residual autocorrelation around the structural prediction. Coefficients and variances are expressions, so they may reference THETA/SIGMA assignments. The generated '\$ERROR' component is inserted automatically by [nm_model()].

Usage

```
nm_arma_config(  
  ar = character(),  
  ma = character(),  
  innovation_variance = "SIGMA(1)",  
  observation_variance = "0",  
  initial_variance = "1000",  
  prefix = "ARMA",  
  by_dvid = TRUE  
)
```

Arguments

ar	Character expressions for AR coefficients.
ma	Character expressions for MA coefficients.
innovation_variance	Expression for innovation variance.
observation_variance	Additional independent measurement variance; zero is valid.
initial_variance	Diffuse initial variance expression.
prefix	Prefix for generated assignment names.

by_dvid

Maintain independent ARMA histories by DVID.

Value

An 'nm_kalman_config' carrying generated '\$ERROR' assignments.

Combine timing evidence with a correctness gate

Description

Combine timing evidence with a correctness gate

Usage

```
nm_benchmark_gate(validation, timings, provenance)
```

Arguments

validation	Result from [nm_validation_gate()].
timings	Named timing summary.
provenance	Result from [nm_benchmark_provenance()].

Value

A benchmark record whose 'publishable' flag is true only after a passing numerical gate and at least three measured repetitions.

Record reproducible benchmark provenance

Description

Record reproducible benchmark provenance

Usage

```
nm_benchmark_provenance(label, repetitions, warmup = 1L, extra = list())
```

Arguments

label	Benchmark label.
repetitions	Number of measured repetitions after warm-up.
warmup	Number of discarded warm-up runs.
extra	Additional named metadata.

Subject-level nonparametric bootstrap

Description

Subject-level nonparametric bootstrap

Usage

```
nm_bootstrap(fit, n = 100L, seed = 20260713L, level = 0.95, ...)
```

Arguments

fit	An 'nm_fit' used as the model and estimation template.
n	Number of bootstrap fits.
seed	RNG seed.
level	Percentile confidence level.
...	Controls passed to [nm_est()].

Value

Bootstrap parameter matrix, convergence flags, and failed-run errors.

Compile a serializable model into a pointer-backed engine

Description

Compile a serializable model into a pointer-backed engine

Usage

```
nm_compile(model)
```

Arguments

`model`

An `[nm_model()]`.

Define an offline differentiable model component

Description

Components are expanded to restricted scalar expression IR and therefore remain inside C++/CppAD during simulation and estimation. Weight/data payloads are immutable, hashed, serializable, and never access a network.

Usage

```
nm_component(  
  name,  
  type = c("dense_nn", "linear_spline", "gaussian_process"),  
  scope = c("pred", "des"),  
  inputs,  
  outputs,  
  weights = NULL,  
  biases = NULL,  
  activation = c("tanh", "softplus", "relu"),  
  knots = NULL,  
  values = NULL,  
  training = NULL,  
  alpha = NULL,  
  lengthscale = 1,  
  variance = 1,  
  mean = 0  
)
```

Arguments

name	Stable component label.
type	"dense_nn", "linear_spline", or "gaussian_process".
scope	Compile into '\$PK/\$PRED', or directly into '\$DES' for a state/time-dependent learned dynamics or correction term.

inputs	Model symbols used as inputs.
outputs	Generated model assignment names.
weights, biases	Dense-network layer matrices and bias vectors.
activation	Hidden-layer activation.
knots, values	Linear-spline knots and values.
training, alpha, lengthscale, variance, mean	Gaussian-process prediction payload.

Generate restricted expression code for a model component

Description

Generate restricted expression code for a model component

Usage

```
nm_component_code(component)
```

Arguments

component	An [nm_component()].
-----------	----------------------

Read and translate a NONMEM control stream

Description

The importer preserves unsupported records and reports them rather than silently discarding them. The supported semantic subset includes INPUT, DATA, SUBROUTINES, MODEL, PK/PRED, DES, ERROR, THETA, diagonal or BLOCK OMEGA, SIGMA, ESTIMATION, COVARIANCE and TABLE records.

Usage

```
nm_control_read(x, strict = TRUE)
```

Arguments

x	A control-stream path or character text.
---	--

strict	If 'TRUE', fail when the translated model cannot be compiled.
--------	---

Value

An 'nm_control_stream' with the translated 'model', execution metadata, preserved raw text, and a compatibility report.

Write a NONMEM control stream

Description

Write a NONMEM control stream

Usage

```
nm_control_write(  
  x,  
  file = NULL,  
  data = NULL,  
  estimation = NULL,  
  covariance = NULL  
)
```

Arguments

x	An 'nm_model', 'NMEngine', or object returned by [nm_control_read()].
file	Optional destination '.ctl' path. If omitted, text is returned.
data	Dataset path and options written to '\$DATA'. The imported record is reused by default for an 'nm_control_stream'.
estimation	Optional '\$ESTIMATION' options.
covariance	Optional '\$COVARIANCE' options; use 'FALSE' to omit it.

Value

Control-stream text, invisibly when written to 'file'.

Covariance step for a fitted model

Description

Covariance step for a fitted model

Usage

```
nm_cov_step(  
  fit,  
  type = c("auto", "hessian", "opg", "sandwich", "r", "s"),  
  tolerance = 1e-08,  
  samples = NULL,  
  seed = NULL,  
  eta_maxit = NULL  
)
```

Arguments

fit	An 'nm_fit'.
type	'hessian'/'r' uses objective curvature, 'opg'/'s' uses the subject-score matrix, and 'sandwich' uses R-inverse S R-inverse. 'auto' prefers a well-conditioned R matrix and falls back to sandwich or S.
tolerance	Positive-definite regularization tolerance.
samples	Target integration budget for IMP/SAEM marginal information. Low-dimensional ETA integrals use a tensor Gauss–Hermite design near this budget; higher-dimensional integrals use this many random-normal samples. The default reuses the IMP fit sample count or uses 200 for SAEM. GQ fits reuse their estimation grid, tensor order or Smolyak level, and point limit.
seed	Common-random-number seed used by the random-normal fallback for IMP/SAEM information.

eta_maxit

Maximum conditional ETA iterations for GQ/IMP/SAEM information.

Value

Covariance, correlation, standard errors, relative standard errors, eigenvalues, and conditioning diagnostics.

Configure a continuous-time hidden Markov model

Description

'nm_cthmm_config()' is the continuous-time counterpart of [nm_hmm_config()]. Off-diagonal entries of 'generator' name transition-rate assignments in '\$ERROR'; diagonal entries are generated as minus the sum of the other entries in their row. At every observation the compiled engine calculates 'exp(Q * DT)' with a CppAD-compatible matrix exponential. This supports irregular observation times, time-varying rates, exact gradients, retrospective smoothing, and Viterbi decoding.

Usage

```
nm_cthmm_config(  
  states,  
  initial,  
  generator,  
  emission,  
  initial_scale = c("probability", "log"),  
  rate_scale = c("rate", "log"),  
  emission_scale = c("likelihood", "log"),  
  by_dvid = TRUE  
)
```

Arguments

states	Unique labels for the hidden states.
initial	Character vector naming one '\$ERROR' initial-state assignment per state.
generator	Square character matrix. Every off-diagonal element names the '\$ERROR' assignment for the corresponding from-state to to-state rate. Diagonal elements are ignored and may be empty or 'NA'.
emission	Character vector naming one state-conditional '\$ERROR' emission assignment per state.

initial_scale

Whether 'initial' contains probability weights or log weights.

rate_scale

Whether off-diagonal generator assignments contain non-negative rates or unconstrained log rates.

emission_scale

Whether 'emission' contains likelihoods/densities or log likelihoods.

by_dvid

Maintain independent state sequences for each 'DVID' within subject.

Value

A validated, serializable continuous-time hidden Markov configuration.

Configure a semi-explicit index-1 DAE

Description

Algebraic variables are ordinary symbols in '\$DES'. The 'ALG' block assigns 'RES(1)', ..., one residual equation per variable. A fixed-iteration Newton solve remains on the CppAD tape, including implicit parameter derivatives.

Usage

```
nm_dae_config(  
    variables,  
    initial = 0,  
    tolerance = 1e-09,  
    maxit = 12L,  
    jacobian_step = 1e-06,  
    sparsity = NULL  
)
```

Arguments

variables	Algebraic variable names.
initial	Initial Newton values, one per variable.
tolerance	Residual convergence tolerance.
maxit	Maximum Newton iterations per derivative evaluation.
jacobian_step	Relative central-difference step for the algebraic Jacobian.
sparsity	Optional logical residual-by-variable sparsity pattern.

Normalize a NONMEM event dataset

Description

Normalize a NONMEM event dataset

Usage

```
nm_dataset(data, expand_addl = TRUE)
```

Arguments

data	Data frame containing at least 'ID' and 'TIME'.
expand_addl	Materialize ADDL/II doses before C++ execution.

Value

An 'nm_dataset' data frame with stable event ordering.

Examples

```
events <- data.frame(  
  ID = 1, TIME = c(0, 1, 2), EVID = c(1, 0, 0),  
  AMT = c(100, 0, 0), DV = c(NA, 4.2, 3.1), MDV = c(1, 0, 0)  
)  
nm_dataset(events)
```

Configure a delay differential-equation model

Description

Delayed states are referenced in '\$DES' as 'LAG(A(i), delay_name)', where 'delay_name' is a positive assignment produced by '\$PK/\$PRED'. The compiled method-of-steps solver uses fixed steps and differentiable linear history interpolation.

Usage

```
nm_dde_config(  
  history = 0,  
  step = 0.05,  
  interpolation = c("linear"),  
  max_steps = 100000L,  
  minimum_delay = NULL  
)
```

Arguments

history	Scalar or state-length numeric history before the first event.
step	Fixed method-of-steps integration step.
interpolation	Currently "linear".
max_steps	Maximum fixed steps per propagation call.
minimum_delay	Optional declared lower bound for every delay. When supplied it must be at least 'step'.

Generate a LibeRation model from a visual diagram

Description

Generated '\$DES' remains ordinary editable model code. The semantic diagram and last generated code are retained in 'GRAPH', so later visual regeneration can be previewed explicitly without silently overwriting manual edits.

Usage

```
nm_diagram_generate(  
  diagram,  
  input = c("ID", "TIME", "EVID", "AMT", "RATE", "II", "SS", "CMT", "DV",  
    "MDV"),  
  output = c("PRED", "IPRED", "CWRES")  
)
```

Arguments

diagram	An [nm_model_diagram()] object.
input	Optional NONMEM-style input-column names.
output	Optional generated run columns.

Value

A serializable [nm_model()].

Preview code generated by a model diagram

Description

Preview code generated by a model diagram

Usage

```
nm_diagram_preview(diagram)
```

Arguments

diagram	An [nm_model_diagram()] object.
---------	---------------------------------

Value

Generated '\$PK/\$PRED', '\$DES', THETA, OMEGA, and SIGMA scaffolds.

Estimate a population pharmacometric model

Description

Deterministic conditional methods use exact CppAD gradients for ETA modes. LAPLACE uses the exact conditional Hessian; FOCE/FOCEI use the interaction-aware Gauss-Newton curvature; FO integrates the first-order Gaussian linearization analytically. GQ integrates the exact joint objective over ETAs with adaptive tensor or Smolyak sparse Gauss–Hermite quadrature. Stochastic methods use the same C++ joint objective and are implemented in the stochastic estimation module. HMC and NUTS use exact joint CppAD gradients with unconstrained parameter transforms, dual-averaged step-size adaptation, and a diagonal mass matrix. NPML and NPAG replace the Gaussian random-effect distribution with a discrete ETA support distribution; NPAG adapts, expands, and prunes that support. Serial FO, FOCE, FOCEI, Laplace, and ITS fits expose a persistent C++ population objective through thin callbacks to R's L-BFGS-B/BFGS optimizer; PSOCK fits retain R coordination across their persistent C++ workers.

Usage

```
nm_est(  
  model,  
  data,  
  method = c("FOCEI", "FOCE", "FO", "LAPLACE", "ITS", "GQ", "IMP", "SAEM",  
"BAYES",  
    "HMC", "NUTS", "NPML", "NPAG"),  
  maxit = 200L,  
  eta_maxit = 100L,  
  tolerance = 1e-06,  
  trace = 0L,  
  print_every = 0L,  
  n_cores = 1L,  
  optimizer_backend = c("auto", "native", "r"),  
  covariance = FALSE,  
  covariance_type = c("auto", "hessian", "opg", "sandwich", "r", "s"),  
  covariance_tolerance = 1e-08,  
  covariance_samples = NULL,  
  covariance_seed = NULL,  
  initial_eta = NULL,  
  collect_output = TRUE,  
  ...  
)
```


Arguments

<code>model</code>	An <code>[nm_model()]</code> or compiled <code>[NMEngine]</code> .
<code>data</code>	A NONMEM-style event dataset.
<code>method</code>	Estimation method.
<code>maxit</code>	Maximum outer evaluations.
<code>eta_maxit</code>	Maximum conditional ETA iterations.
<code>tolerance</code>	Relative optimization tolerance.
<code>trace</code>	Optimizer tracing level.
<code>print_every</code>	Write the objective and scaled population gradient to stdout every N outer evaluations. Zero disables iteration logging.
<code>n_cores</code>	Number of parallel subject workers. Parallel workers persist compiled C++ engines for the duration of the estimation.
<code>optimizer_backend</code>	Use the experimental compiled scaled, bounded BFGS optimizer (<code>"native"</code>), R's mature L-BFGS-B/BFGS driver (<code>"r"</code>), or the calibrated production policy (<code>"auto"</code>). Auto selects R's driver while its objective, gradient, conditional-mode state, parameter transforms, and covariance derivatives remain in the persistent C++ population evaluator.
<code>covariance</code>	Run a covariance step after estimation.

<code>covariance_type</code>	Covariance estimator: automatic R/S fallback, objective Hessian ("hessian"), subject-score OPG ("opg"), or robust sandwich ("sandwich"). "r" and "s" are accepted aliases.
<code>covariance_tolerance</code>	Positive-definite regularization tolerance for the covariance step.
<code>covariance_samples</code>	Target integration budget for an IMP or SAEM covariance step. Low-dimensional ETA integrals use tensor Gauss–Hermite quadrature near this budget; higher-dimensional integrals use random-normal importance samples. The default reuses the IMP count or uses 200 for SAEM.
<code>covariance_seed</code>	Common-random-number seed for the random-normal covariance fallback. The default reuses the estimation seed.
<code>initial_eta</code>	Optional finite subject-by-ETA matrix used to warm-start compatible conditional or SAEM estimation steps.
<code>collect_output</code>	Whether selected generated OUTPUT columns should be evaluated and retained with the completed fit.
<code>...</code>	Method-specific controls. For 'method = "GQ"', use 'gq_grid' ("auto", "tensor", or "smolyak"), 'gq_order' (tensor nodes per ETA dimension, default 5), 'gq_level' (Smolyak level, default 3), 'gq_adaptive' (default 'TRUE'), 'gq_max_points' (retained-grid allocation guard, default 100000), and 'gq_gradient' ("score" or the slower "finite_grid"). Automatic selection uses tensor quadrature through three ETAs and Smolyak quadrature for higher-dimensional models. For 'method = "HMC" or "NUTS"', controls include 'n_warmup' (500), 'n_sample' (1000 per chain), 'n_thin' (1), 'n_chains' (4), 'seed', optional 'step_size', 'target_acceptance' (0.8), 'adapt_mass' ('TRUE'), 'n_leapfrog' (10; HMC), 'max_depth' (10; NUTS), and 'divergence_threshold' (1000). For 'method = "NPML" or "NPAG"', use 'np_supports' for an optional fixed starting matrix, 'np_points' (25), 'np_max_support' (100), 'np_min_weight' (1e-5), 'np_weight_maxit' (1000), 'np_cycles'

(3), and, for NPAG, 'np_grid_step' (1), 'np_grid_decay' (0.5), and 'np_max_candidates' (500). 'np_estimate_population' controls alternating THETA/SIGMA updates. Ordinary covariance is not regular for a discrete support distribution; use bootstrap uncertainty for NPML/NPAG.

Run multiple estimation methods sequentially

Description

Population estimates from each completed step become the initial THETA, OMEGA, and SIGMA values for the next. Compatible conditional ETA estimates are used as a warm start for FOCE/FOCEI/LAPLACE and SAEM. The returned object remains an 'nm_fit' for compatibility and contains compact per-step results in 'stages' plus 'method_sequence' and 'sequence_label' metadata.

Usage

```
nm_est_sequence(model, data, stages, on_error = c("stop", "return"))
```

Arguments

model	An [nm_model()] or compiled [NMEngine()].
data	NONMEM-style event data.
stages	Ordered character vector or list of [nm_est_stage()] objects.
on_error	Stop with an 'nm_est_sequence_error', or return the last successful fit with failure metadata.

Value

The final 'nm_fit', augmented with sequential-estimation metadata.

Define one step in a sequential estimation

Description

Define one step in a sequential estimation

Usage

```
nm_est_stage(method, ..., label = NULL)
```

Arguments

method	LibeRation estimation method.
...	Controls passed to [nm_est()] for this step.
label	Optional human-readable step label.

Value

A serializable 'nm_est_stage' specification.

Empirical Bayes estimates and shrinkage

Description

Empirical Bayes estimates and shrinkage

Usage

```
nm_etab(fit)
```

Arguments

fit	An 'nm_fit'.
-----	--------------

Value

A list with subject ETA table and component-wise shrinkage.

Configure experimental engine behaviour

Description

Experimental models retain their approximation, reproducibility, and provenance metadata in the serialized model and every queued job.

Usage

```
nm_experimental_config(enabled = TRUE, label = NULL, strict = TRUE)
```

Arguments

enabled	Explicitly acknowledge use of an experimental engine.
label	Optional user-facing purpose or validation label.
strict	Fail rather than fall back to a less exact algorithm.

Value

Serializable experimental-engine metadata.

Define one latent chain in a factorial HMM

Description

Define one latent chain in a factorial HMM

Usage

```
nm_factorial_chain(  
  name,  
  states,  
  initial,  
  transition,  
  initial_scale = c("probability", "log"),  
  transition_scale = c("probability", "log")  
)
```

Arguments

name	Chain label.
states	State labels.
initial	Initial-weight assignment names.
transition	Row-by-column transition assignment names.
initial_scale, transition_scale	Probability or log scale.

Configure an exact factorial hidden Markov model

Description

Independent chain transitions are combined lazily into one sparse-compatible joint-state HMM. State-conditional emission assignments are supplied for the Cartesian joint states. Decoding retains joint probabilities and adds chain marginals.

Usage

```
nm_factorial_hmm_config(  
  ...,  
  emission,  
  emission_scale = c("likelihood", "log"),  
  by_dvid = TRUE,  
  max_joint_states = 256L,  
  prefix = "FHMM"  
)
```

Arguments

...	[nm_factorial_chain()] declarations.
emission	One emission assignment per joint state.
emission_scale	Likelihood or log scale.
by_dvid	Maintain independent sequences by DVID.
max_joint_states	Safety limit for exact enumeration.
prefix	Generated assignment prefix.

Goodness-of-fit table

Description

Computes population/individual predictions and residuals on the estimation records. 'WRES' and 'IWRES' use the configured residual variance. 'CWRES' are decorrelated within subject using the exact AD prediction Jacobian and a first-order conditional ETA covariance. Censored observations remain marked and receive 'NA' residual diagnostics.

Usage

```
nm_gof(fit)
```

Arguments

`fit`

An 'nm_fit'.

Value

A data frame aligned to the fitted event data.

Configure a finite-state hidden Markov model

Description

'nm_hmm_config()' maps assignments in a model's '\$ERROR' block to the initial-state weights, transition weights, and state-conditional emission likelihoods used by LibeRation's compiled forward algorithm. Initial and transition weights are normalized internally, so users may supply either probabilities or positive unnormalised weights. Log weights are normalized with a stable softmax. Emissions may be probabilities, densities, or log likelihoods.

Usage

```
nm_hmm_config(  
  states,  
  initial,  
  transition,  
  emission,  
  initial_scale = c("probability", "log"),  
  transition_scale = c("probability", "log"),  
  emission_scale = c("likelihood", "log"),  
  by_dvid = TRUE  
)
```

Arguments

states	Unique labels for the hidden states.
initial	Character vector naming one '\$ERROR' assignment per state.
transition	Character matrix naming '\$ERROR' assignments for each from-state (row) to-state (column) transition.
emission	Character vector naming one state-conditional '\$ERROR' emission assignment per state.

<code>initial_scale</code>	Whether 'initial' assignments contain non-negative weights or log weights.
<code>transition_scale</code>	Whether 'transition' assignments contain non-negative weights or log weights.
<code>emission_scale</code>	Whether 'emission' assignments contain positive likelihoods/densities or log likelihoods.
<code>by_dvid</code>	Maintain independent hidden-state sequences for each 'DVID' within subject. If false, all observations within subject form one sequence.

Value

A validated, serializable hidden Markov configuration.

Decode hidden states

Description

Runs the same scaled forward algorithm used by estimation and optionally a retrospective scaled forward-backward smoother or log-domain Viterbi decoder. Filtering conditions on observations up to the current record, smoothing conditions on the complete sequence, and Viterbi returns the single most probable joint state path. All results are conditional on the supplied or fitted parameters and ETAs.

Usage

```
nm_hmm_decode(  
  object,  
  data = NULL,  
  type = c("individual", "population"),  
  theta = NULL,  
  eta = NULL,  
  sigma = NULL,  
  method = c("filtered", "smoothed", "viterbi", "all")  
)
```

Arguments

object	An [nm_model()], compiled [NMEngine], or fitted 'nm_fit'.
data	Observation data. May be omitted for an 'nm_fit' to use its estimation dataset.
type	For fitted models, use individual ETAs or population ETAs.
theta, eta, sigma	Optional parameters for an unfitted model.
method	Decoding result to return. "'filtered'", "'smoothed'", and "'viterbi'" expose the selected state through 'HMM_STATE'; "'all'" returns explicitly prefixed columns for all three decoders.

Value

The input data augmented with state, probability, and 'HMM_ROW_NLL' columns. The total log likelihood and per-sequence Viterbi summaries are stored in the 'log_likelihood' and 'sequence_summary' attributes.

Configure a discrete hidden semi-Markov model

Description

Expands explicit state-duration distributions into a sparse finite-state representation consumed by the compiled HMM forward, smoother, Viterbi, and automatic-differentiation paths. Duration is measured in observation steps.

Usage

```
nm_hsmm_config(  
  states,  
  initial,  
  transition,  
  dwell,  
  emission,  
  initial_scale = c("probability", "log"),  
  transition_scale = c("probability", "log"),  
  dwell_scale = c("probability", "log"),  
  emission_scale = c("likelihood", "log"),  
  by_dvid = TRUE,  
  prefix = "HSMM"  
)
```

Arguments

states	Original semi-Markov state labels.
initial, emission	One '\$ERROR' assignment per state.
transition	Square matrix of between-state transition assignments.
dwell	Matrix with one row per state and one column per possible dwell duration.

initial_scale, transition_scale,
dwell_scale, emission_scale

Probability or log scales.

by_dvid

Maintain independent sequences by DVID.

prefix

Prefix for generated expanded-state
assignments.

Fit one patient's individual random effects

Description

Estimates a patient's ETA vector while holding the population model and its THETA, OMEGA, and SIGMA values fixed. The conditional objective and its derivatives are evaluated by the same persistent C++/CppAD engine used by population estimation. A non-zero Gaussian prior permits sequential and dynamic-state updating without treating an earlier point estimate as exact.

Usage

```
nm_individual_fit(  
  model,  
  data,  
  theta = NULL,  
  sigma = NULL,  
  omega = NULL,  
  start = NULL,  
  prior_mean = NULL,  
  prior_covariance = NULL,  
  maxit = 100L,  
  tolerance = 1e-07,  
  interaction = TRUE,  
  exact_hessian = TRUE  
)
```

Arguments

model	An [nm_model()] or compiled [NMEngine].
data	A one-subject NONMEM-style event dataset.
theta, sigma, omega	Fixed population parameter values.
start	Initial ETA vector.

prior_mean	Optional Gaussian prior mean for the expanded ETA vector.
prior_covariance	Optional Gaussian prior covariance. Supplying either custom-prior argument requires both.
maxit	Maximum conditional-mode iterations.
tolerance	Relative optimizer tolerance.
interaction	Include ETA-residual interaction.
exact_hessian	Use the exact conditional Hessian.

Value

An 'nm_individual_fit' containing the MAP ETA state, Laplace covariance, predictions, diagnostics, and the fixed population values.

Declare an item-response/ordered-categorical endpoint set

Description

This convenience constructor creates one ordinal [nm_outcome()] per item and combines them by 'DVID'. Item characteristic curves remain ordinary named '\$PK/\$PRED' probability assignments, so graded-response, partial-credit, or custom IRT parameterizations can all use the same compiled engine.

Usage

```
nm_irt_outcomes(item_dvid, probabilities, categories = NULL, names = NULL)
```

Arguments

item_dvid	Numeric DVID code for each item.
probabilities	List containing one character probability vector per item.
categories	Shared category codes or a list of codes per item.
names	Optional item labels.

Value

An 'nm_outcomes' declaration.

Configure a Gaussian state-space model

Description

Defines a continuous-observation linear or nonlinear Gaussian state-space likelihood. Linear models use an exact Kalman filter. Nonlinear transition and observation assignments can use an extended Kalman filter (EKF), unscented Kalman filter (UKF), or a seeded bootstrap particle filter. All deterministic filter operations remain in the compiled C++ objective. Every name refers to an assignment in '\$ERROR', so transition, process, observation, and covariance terms may depend on 'THETA', 'ETA', covariates, 'DT', and ordinary '\$PK/\$PRED' outputs. The compiled C++ engine runs the Kalman filter as part of the exact CppAD population objective and exposes a Rauch–Tung–Striebel retrospective smoother.

Usage

```
nm_kalman_config(  
  states,  
  initial_mean,  
  initial_covariance,  
  transition,  
  process_covariance,  
  observation,  
  observation_variance,  
  baseline = c("prediction", "zero"),  
  by_dvid = TRUE,  
  filter = c("linear", "ekf", "ukf", "particle"),  
  state_inputs = NULL,  
  jacobian_step = 1e-05,  
  ukf_alpha = 0.5,  
  ukf_beta = 2,  
  ukf_kappa = 0,  
  particles = 256L,  
  ess_threshold = 0.5,  
  seed = 20260721L,  
  dynamics = c("discrete", "sde"),  
  sde_method = c("euler", "milstein"),  
  sde_substeps = 8L  
)
```

Arguments

states	Unique labels for latent continuous states.
initial_mean	Character vector naming the initial-state means.
initial_covariance	Square symmetric character matrix naming the initial covariance entries.
transition	For 'filter = "linear"', a square character matrix naming the discrete transition matrix entries. For a nonlinear filter, a character vector naming the next-state function for every state.
process_covariance	Square symmetric character matrix naming process covariance entries for the current interval.
observation	For 'filter = "linear"', a character vector naming the observation-loading row 'H'. For a nonlinear filter, one assignment naming the scalar observation function.
observation_variance	One assignment naming the positive measurement variance.
baseline	Add the structural prediction 'F' to the latent-state observation mean, or use a zero baseline.
by_dvid	Maintain a separate latent trajectory for each DVID within subject.
filter	State inference algorithm: exact linear Kalman, EKF, UKF, or seeded bootstrap particle filtering.
state_inputs	Input symbols used for latent states by nonlinear transition and observation assignments. The default is 'STATE_<name>'.

jacobian_step	Relative central-difference step used for EKF state Jacobians. Parameter derivatives are still propagated by CppAD.
ukf_alpha, ukf_beta, ukf_kappa	Scaled unscented-transform controls.
particles	Number of particles for the bootstrap filter.
ess_threshold	Resample when effective sample size falls below this proportion of 'particles'.
seed	Reproducible common-random-number seed for particle inference.
dynamics	Discrete transition assignments or continuous-discrete SDE drift/diffusion assignments. Prefer [nm_sde_config()] for SDE models.
sde_method	Fixed-step Euler–Maruyama or diagonal Milstein propagation for SDE dynamics.
sde_substeps	Number of fixed SDE steps per observation interval.

Details

The measurement equation is ' $DV = F + H x + \text{epsilon}$ ' when 'baseline' is "'prediction'", and ' $DV = H x + \text{epsilon}$ ' when it is "'zero'".

Value

A validated serializable linear Gaussian state-space declaration.

Filter and smooth latent Gaussian states

Description

Filter and smooth latent Gaussian states

Usage

```
nm_kalman_decode(  
  object,  
  data = NULL,  
  type = c("individual", "population"),  
  theta = NULL,  
  eta = NULL,  
  sigma = NULL  
)
```

Arguments

object	An [nm_model()], compiled [NMEngine], or fitted 'nm_fit' with 'KALMAN_CONFIG'.
data	Observation data; defaults to fitted data for an 'nm_fit'.
type	Use individual or population ETAs for a fitted model.
theta, eta, sigma	Optional parameters for an unfitted model.

Value

Observation data augmented with filtered, predicted, and smoothed latent-state means and standard deviations, innovations, innovation variances, and row likelihood contributions.

Configure population likelihood features

Description

Configure population likelihood features

Usage

```
nm_lik_config(  
  error = c("auto", "none", "combined", "additive", "proportional",  
"exponential",  
  "power", "likelihood"),  
  omega = c("diagonal", "full"),  
  sigma_corr = c("independent", "ar1"),  
  sigma_parameterization = c("sd", "variance"),  
  ar1_rho = 0,  
  ar1_parameter = NULL,  
  ar1_transform = c("tanh", "identity"),  
  blk_method = c("none", "m3", "m4"),  
  lloq = NA_real_,  
  iov = 0L,  
  occasion_col = "OCC",  
  residual_groups = NULL,  
  priors = NULL,  
  mixtures = NULL  
)
```

Arguments

error

Residual model. 'auto' uses the model's 'ERROR' block. 'likelihood' compiles a user-defined row contribution from 'LIK' (a probability or density) or 'LOGLIK' (a log likelihood) in '\$ERROR'.

omega

Random-effect covariance structure.

sigma_corr

Independent or within-subject AR(1) residual errors.

<code>sigma_parameterization</code>	Interpret SIGMA values as residual standard deviations (the historical LibeRation convention) or variances (the native NONMEM '\$SIGMA' convention).
<code>ar1_rho</code>	Fixed AR(1) correlation in (-1, 1). Ignored when 'ar1_parameter' is supplied.
<code>ar1_parameter</code>	Optional estimated AR(1) parameter written as "THETA(i)" or "SIGMA(i)".
<code>ar1_transform</code>	Transform applied to 'ar1_parameter'. The default hyperbolic tangent maps an unconstrained parameter exactly into '(-1, 1)'; "identity" expects the parameter itself to remain strictly inside that interval.
<code>blq_method</code>	No censoring, Beal M3, or Beal M4.
<code>lloq</code>	Optional scalar LLOQ used when a row-specific 'LLOQ' column is absent.
<code>iov</code>	Number of trailing ETA definitions that are occasion-specific.
<code>occasion_col</code>	Dataset column identifying occasions.
<code>residual_groups</code>	Optional correlated endpoint declarations created by [nm_residual_group()].
<code>priors</code>	Optional fixed-effect prior definitions.
<code>mixtures</code>	Optional mixture-model definitions.

Value

A validated serializable likelihood configuration.

Define an arbitrary linear compartment graph

Description

Define an arbitrary linear compartment graph

Usage

```
nm_matrix_model(  
  compartments,  
  flows,  
  observations = NULL,  
  inputs = NULL,  
  layout = NULL  
)
```

Arguments

compartments	Data frame with stable 'id' and 'name' columns.
flows	Data frame describing directed flows and their parameterization.
observations	Observation definitions.
inputs	Dosing/input definitions.
layout	Optional diagram layout kept separate from semantics.

Define a finite mixture model

Description

The model's PRED block may refer to 'MIXNUM' in tape-safe 'ifelse()' expressions. Subject likelihoods are combined with the supplied prior component probabilities using an exact log-sum-exp calculation.

Usage

```
nm_mixture(probability, label = NULL)
```

Arguments

probability	Positive component probabilities.
-------------	-----------------------------------

label	Optional component labels.
-------	----------------------------

Value

A serializable mixture definition for 'nm_lik_config()'.

Posterior subject mixture probabilities

Description

Posterior subject mixture probabilities

Usage

```
nm_mixture_posterior(  
  model,  
  data = NULL,  
  theta = NULL,  
  eta = NULL,  
  sigma = NULL  
)
```

Arguments

model	An 'nm_model', 'NMEngine', or fitted 'nm_fit'.
data	Event data; defaults to fitted data for an 'nm_fit'.
theta, eta, sigma	Parameter values; fit values are used when available.

Value

One row per subject with component probabilities and most likely class.

Define a NONMEM-style pharmacometric model

Description

The established LibeRation 'PRED' and 'ERROR' strings are retained. They are compiled to C++ expression IR; they are not evaluated inside the numerical loop by R.

Usage

```
nm_model(  
  INPUT,  
  OUTPUT = NULL,  
  ADVAN = 2L,  
  TRANS = 2L,  
  SS = 0L,  
  DOSECMP = 1L,  
  OBSCMP = 1L,  
  PRED = "",  
  ERROR = "Y = F",  
  DES = "",  
  ALG = "",  
  THETAS,  
  OMEGAS = NULL,  
  SIGMAS = NULL,  
  COVARIATES = NULL,  
  USE_ODE = FALSE,  
  ODE_CONTROL = NULL,  
  IOV = 0L,  
  LIK_CONFIG = NULL,  
  HMM_CONFIG = NULL,  
  KALMAN_CONFIG = NULL,  
  DDE_CONFIG = NULL,  
  DAE_CONFIG = NULL,  
  RE_CONFIG = NULL,  
  COMPONENTS = NULL,  
  EXPERIMENTAL = NULL,  
  OUTCOMES = NULL,  
  SOLVER = c("auto", "advan", "matrix", "ode", "dde", "dae"),  
  ERROR_TYPE = c("auto", "none", "additive", "proportional", "combined",  
    "exponential",  
    "power", "likelihood"),
```

```

GRAPH = NULL,
LAYOUT = NULL,
LANGUAGE = c("R", "C++")
)

```

Arguments

INPUT	Required dataset column names.
OUTPUT	Optional generated run columns. Available names are discovered from '\$PK/\$PRED' assignments and combined with standard engine outputs; see [nm_model_outputs()].
ADVAN	ADVAN number.
TRANS	TRANS parameterization number.
SS	Model-level steady-state default.
DOSECMP	Default dosing compartment.
OBSCMP	Default observation compartment.
PRED	Parameter/model assignment code.
ERROR	Residual-error or user-likelihood assignment code. With 'LIK_CONFIG = nm_lik_config(error = "likelihood")', assign either 'LIK' (a positive row probability/density) or 'LOGLIK' (its logarithm). The compiled block may use 'DV', 'F'/'PRED'/'IPRED', model assignments, parameters, numeric input columns, and the Markov helpers 'PREV_DV', 'PREV_TIME', 'DT', and 'FIRST'.

DES	ODE derivative code for ADVAN6/13.
ALG	Algebraic residual equations for an experimental index-1 DAE.
THETAS, OMEGAS, SIGMAS	Parameter tables.
COVARIATES	Dataset covariates exposed to 'PRED'.
USE_ODE	Whether an ODE solver is explicitly requested.
ODE_CONTROL	Named list with 'rtol', 'atol', 'max_steps', and optional 'initial_step'. ADVAN6 uses adaptive Dormand-Prince 5(4); ADVAN13 uses an A-stable adaptive implicit trapezoidal method.
IOV	Number of trailing inter-occasion ETAs.
LIK_CONFIG	Reserved likelihood configuration.
HMM_CONFIG	Optional finite-state hidden Markov configuration created by [nm_hmm_config()] or [nm_cthmm_config()]. Its initial, transition/rate, and emission names refer to assignments in 'ERROR'; the complete scaled forward likelihood runs in C++ and remains differentiable by CppAD.
KALMAN_CONFIG	Optional linear Gaussian state-space configuration created by [nm_kalman_config()]. Filtering is part of the compiled exact likelihood; retrospective smoothing is available through [nm_kalman_decode()].
DDE_CONFIG	Optional method-of-steps declaration created by [nm_dde_config()]. Delayed states use 'LAG(A(i), delay_name)' in 'DES'.

DAE_CONFIG	Optional semi-explicit index-1 declaration created by [nm_dae_config()]. Algebraic residuals are supplied through 'ALG'.
RE_CONFIG	Optional nested/crossed random-effect design created by [nm_re_config()]. Every ETA is assigned to a grouping block; connected components are evaluated as independent conditional-likelihood units.
COMPONENTS	Optional list of immutable offline [nm_component()] declarations expanded into compiled '\$PK/\$PRED' or '\$DES' IR according to each component's scope.
EXPERIMENTAL	Explicit experimental-engine acknowledgement and policy.
OUTCOMES	Optional first-class observation declaration created by [nm_outcome()] or [nm_outcomes()]. When 'ERROR' is omitted, LibeRation generates an editable normalized likelihood block. The declaration is also used for stochastic outcome simulation and family diagnostics.
SOLVER	'auto', 'advan', 'matrix', 'ode', 'dde', or 'dae'.
ERROR_TYPE	Standard residual-error form, or 'auto'.
GRAPH	Optional semantic compartment graph.
LAYOUT	Optional graphical layout, stored separately from 'GRAPH'.
LANGUAGE	Model source language ('R' or restricted 'C++').

Value

A serializable 'nm_model'.

Examples

```
model <- nm_model(  
  INPUT = c("ID", "TIME", "EVID", "AMT", "CMT", "DV", "MDV"),  
  ADVAN = 1,  
  PRED = "CL=THETA(1)*exp(ETA(1)); V=THETA(2); S1=V",  
  ERROR = "Y=F*(1+ERR(1))",  
  THETAS = data.frame(THETA = 1:2, Value = c(2, 20)),  
  OMEGAS = data.frame(OMEGA = 1, Value = 0.1),  
  SIGMAS = data.frame(SIGMA = 1, Value = 0.1)  
)  
model
```

Versioned semantic model contracts

Description

These helpers are the single source of truth for transporting a model between LibeR packages. Compiled IR and external pointers are deliberately excluded; the receiving process recompiles the semantic model definition.

Usage

```
nm_model_contract_fields(version = 2L)

nm_model_to_contract(model, version = 2L)

nm_model_from_contract(contract)
```

Arguments

version	Contract version. Version 1 is retained for compatibility; version 2 contains every first-class model configuration in LibeRation.
model	An [nm_model()] object.
contract	A contract returned by [nm_model_to_contract()].

Value

Character vector of semantic model fields.

'nm_model_to_contract()' returns a JSON-compatible semantic contract.

'nm_model_from_contract()' returns a newly validated and compiled 'nm_model'.

Define a model using the restricted C++ expression form

Description

The first compiler version accepts scalar C++-style assignment expressions using the same 'THETA(i)'/ 'ETA(i)' calls as the R form. Arbitrary headers, allocation, I/O, and side effects are intentionally not accepted.

Usage

```
nm_model_cpp(CPP, ...)
```

Arguments

CPP	Parameter/model assignment code.
...	Other arguments passed to [nm_model()].

Define a visual PK/PD model diagram

Description

The diagram is a serializable semantic representation. It supports linear rate and clearance flows, Michaelis-Menten elimination, zero-order source or sink terms, and arbitrary nonlinear flux expressions. Layout is kept alongside, but separate from, the generated model code.

Usage

```
nm_model_diagram(  
  compartments,  
  flows = NULL,  
  parameters = NULL,  
  advan = 6L,  
  residual = c("additive", "proportional", "combined"),  
  covariates = NULL,  
  title = "Diagram model"  
)
```

Arguments

compartments	Data frame or list of compartment rows.
flows	Data frame or list of flow rows.
parameters	Optional parameter settings. Missing referenced parameters are added with positive initials and log-normal IIV.
advan	ODE solver, ADVAN6 or ADVAN13.
residual	Residual-error scaffold: additive, proportional, or combined.
covariates	Optional dataset covariates available to custom fluxes.

title

Diagram/model title.

Value

A serializable 'nm_model_diagram'.

Recover a visual diagram from a generated model

Description

Recover a visual diagram from a generated model

Usage

```
nm_model_diagram_get(model)
```

Arguments

<code>model</code>	An [nm_model()] or [NMEngine].
--------------------	--------------------------------

Value

The stored 'nm_model_diagram', or 'NULL' for a code-only model.

List selectable generated model-run columns

Description

This is a static catalogue produced while '\$PK/\$PRED' is compiled. It does not execute the model or require a dataset. Standard engine outputs are included alongside named assignment results.

Usage

```
nm_model_outputs(model)
```

Arguments

<code>model</code>	An <code>[nm_model()]</code> or compiled <code>[NMEngine()]</code> .
--------------------	--

Value

A data frame with output name, source, run availability, description, and selection status.

Create an editable advanced structural model

Description

These are ordinary [nm_model()] objects, not a second modelling language. The generated '\$PK/\$PRED' and '\$DES' blocks remain fully editable and run through the same C++/CppAD ODE, likelihood, simulation, and estimation paths.

Usage

```
nm_model_template(  
  template = c("nonlinear_elimination", "transit_absorption",  
    "dual_absorption",  
    "parent_metabolite", "effect_compartment", "indirect_response",  
    "tumour_growth",  
    "tmdd"),  
  iiv = TRUE,  
  residual = c("proportional", "additive", "combined", "lognormal",  
    "none"),  
  n_transit = 3L,  
  ode_control = NULL  
)
```

Arguments

template	Template identifier from [nm_structural_templates()].
iiv	Add log-normal ETA variability to generated parameters.
residual	Residual model.
n_transit	Number of transit compartments for 'transit_absorption'.
ode_control	Optional ADVAN13 solver controls.

Value

An editable 'nm_model' with template notes attached.

Numerical predictive check

Description

Computes the empirical predictive percentile of every uncensored observation under repeated simulations from the fitted population model.

Usage

```
nm_npc(fit, nsim = 200L, seed = 20260713L)
```

Arguments

<code>fit</code>	An 'nm_fit'.
<code>nsim</code>	Number of predictive simulations; at least 20.
<code>seed</code>	Reproducible RNG seed.

Value

An 'nm_npc' object with record-level percentiles and tail flags.

Normalized prediction distribution errors

Description

Within each subject, simulations are centered and decorrelated with their empirical predictive covariance before record-wise predictive ranks are transformed to standard-normal NPDE values.

Usage

```
nm_npde(fit, nsim = 200L, seed = 20260713L, ridge = 1e-08)
```

Arguments

fit	An 'nm_fit'.
nsim	Number of predictive simulations; at least 20.
seed	Reproducible RNG seed.
ridge	Relative covariance ridge used for numerically singular blocks.

Value

An 'nm_npde' object with record-level NPDE and summary moments.

Evaluate the exact C++ population joint objective

Description

The objective is minus twice the observation log likelihood plus the ETA Gaussian prior. The complete prediction and likelihood calculation is recorded by CppAD, so gradients and Hessians do not use finite differences.

Usage

```
nm_objective(  
  model,  
  data,  
  theta = NULL,  
  eta = NULL,  
  sigma = NULL,  
  omega = NULL,  
  gradient = TRUE,  
  hessian = FALSE  
)
```

Arguments

model	An 'nm_model' or compiled 'NMEngine'.
data	NONMEM-style event data containing 'DV' and 'MDV'.
theta, eta, sigma, omega	Parameter values.
gradient	Return the exact gradient.
hessian	Return the exact Hessian.

Value

Objective value and requested derivatives.

Declare a first-class pharmacometric outcome

Description

'nm_outcome()' describes an observation distribution independently of the structural PK/PD model. LibeRation translates the declaration into editable '\$ERROR' code, compiles it with CppAD, and retains enough metadata for stochastic simulation and outcome-appropriate diagnostics. Several declarations can be combined with [nm_outcomes()] and selected by 'DVID'.

Usage

```
nm_outcome(  
  family = c("normal", "lognormal", "student_t", "bernoulli",  
"categorical", "ordinal",  
  "poisson", "negative_binomial", "binomial", "zero_inflated_poisson",  
  "hurdle_poisson", "tte", "recurrent_event", "competing_risks",  
"markov",  
  "continuous_time_markov"),  
  name = NULL,  
  dvid = NULL,  
  prediction = "F",  
  scale = NULL,  
  probabilities = NULL,  
  categories = NULL,  
  trials = NULL,  
  dispersion = NULL,  
  zero_probability = NULL,  
  event = 1,  
  cause_hazards = NULL,  
  initial = NULL,  
  transition = NULL,  
  rates = NULL,  
  df = 4,  
  max_count = 100L,  
  risk = NULL  
)
```

Arguments

family	Observation family. Supported families are Gaussian, log-normal, fixed-degrees-of-freedom Student t, Bernoulli, categorical, ordinal, Poisson, negative binomial, binomial, zero-inflated Poisson, hurdle Poisson, first/recurrent time-to-event, competing risks, and observed finite-state Markov models, and exact two-state continuous-time Markov models.
name	Human-readable endpoint name.
dvid	Optional numeric 'DVID' selecting this endpoint. It is required when several outcomes are combined.
prediction	Name of a '\$PK/\$PRED' assignment containing the mean, probability, intensity, or hazard. 'F' uses the engine prediction.
scale	Residual SD for continuous families, as a number, a named assignment, or 'THETA(i)'/ 'SIGMA(i)'.
probabilities	Named '\$PK/\$PRED' probability assignments for categorical or ordinal outcomes.
categories	Numeric observed category codes corresponding to 'probabilities'.
trials	Number of binomial trials, either fixed or supplied by a numeric dataset/model symbol.
dispersion	Negative-binomial size parameter.
zero_probability	Structural-zero probability for ZIP/hurdle models.
event	Numeric event code for TTE models.

<code>cause_hazards</code>	Named hazard assignments for competing risks; names are the numeric cause codes.
<code>initial</code>	Initial-state probability assignments for Markov models.
<code>transition</code>	Square matrix of transition-probability assignments for Markov models; rows are previous states and columns are current states.
<code>rates</code>	Two named off-diagonal intensity assignments ('q01', 'q10') for a two-state continuous-time Markov model.
<code>df</code>	Fixed Student-t degrees of freedom.
<code>max_count</code>	Maximum supported count used to unroll exact factorial and negative-binomial normalizing terms on the AD tape.
<code>risk</code>	Optional at-risk indicator/expression for event models.

Value

An 'nm_outcome' declaration.

Outcome-appropriate diagnostics

Description

Computes endpoint predictions, conditional variance, observed-category probability or hazard, Pearson/deviance residuals, Brier/log scores, and event-model cumulative hazard from a model declared with `[nm_outcome()]`. Unlike Gaussian CWRES, these quantities retain their natural interpretation for categorical, count, joint, Markov, and event-time outcomes.

Usage

```
nm_outcome_diagnostics(fit, predictions = NULL)
```

Arguments

<code>fit</code>	An 'nm_fit' with first-class 'OUTCOMES'.
<code>predictions</code>	Optional individual prediction table, used internally to avoid repeating model propagation.

Value

A data frame aligned to the estimation records.

Combine endpoint declarations

Description

Combine endpoint declarations

Usage

```
nm_outcomes(...)
```

Arguments

... [nm_outcome()] objects or lists containing them.

Value

An 'nm_outcomes' list.

Generate a piecewise-constant model expression

Description

Generate a piecewise-constant model expression

Usage

```
nm_piecewise(time = "TIME", knots, values)
```

Arguments

time	Symbol containing time, normally 'TIME'.
knots	Increasing interval boundaries.
values	One value/expression per interval ($\text{length}(\text{knots}) + 1$).

Value

Editable nested 'ifelse()' code for '\$PK/\$PRED' or '\$ERROR'.

Exact derivatives of model predictions

Description

Records the complete analytical event/ADVAN/matrix path with CppAD and returns derivatives with respect to THETA, every subject ETA, and SIGMA. Dataset values and event times are treated as fixed inputs. ADVAN6 and ADVAN13 record their accepted adaptive trajectories—including periodic steady-state shooting—on the same persistent CppAD tape.

Usage

```
nm_prediction_derivatives(  
  model,  
  data,  
  theta = NULL,  
  eta = NULL,  
  sigma = NULL,  
  jacobian = TRUE  
)
```

Arguments

model	An 'nm_model' or compiled 'NMEngine'.
data	NONMEM-style event data.
theta	Population fixed effects.
eta	Subject ETA matrix.
sigma	Residual parameters.
jacobian	Whether to return the full prediction Jacobian.

Value

A list with 'value', 'jacobian', and differentiation 'domain'.

Define parameter priors

Description

Define parameter priors

Usage

```
nm_prior(  
  parameter,  
  distribution = c("normal", "lognormal", "half_normal", "inverse_gamma"),  
  mean = 0,  
  sd = 1,  
  shape = NA_real_,  
  rate = NA_real_  
)
```

Arguments

parameter	Parameter name such as 'THETA1', 'SIGMA1', or 'OMEGA1'.
distribution	Prior family.
mean, sd	Normal/log-normal parameters.
shape, rate	Inverse-gamma parameters.

Value

A one-row prior table suitable for 'nm_lik_config(priors=...)'.

Profile-likelihood parameter uncertainty

Description

Each grid point fixes one native-scale parameter and re-estimates every remaining free parameter with the fit's estimation method. Confidence limits use the likelihood-ratio cutoff on the '-2 log likelihood' objective scale.

Usage

```
nm_profile(fit, parameters = NULL, points = 9L, span = 3, level = 0.95, ...)
```

Arguments

fit	An 'nm_fit'.
parameters	Native parameter names such as 'THETA1' or 'SIGMA1'.
points	Odd number of grid points per parameter.
span	Half-width in standard errors (or fallback parameter scales).
level	Confidence level.
...	Controls passed to [nm_est()].

Value

An 'nm_profile' containing grid fits and confidence intervals.

Copy a project snapshot into a new model version

Description

Copy a project snapshot into a new model version

Usage

```
nm_project_copy(  
  workspace,  
  project,  
  snapshot = "latest",  
  target_project = project,  
  label = NULL  
)
```

Arguments

workspace	Workspace object or path.
project	Source project id.
snapshot	Source snapshot id or "latest".
target_project	Destination project id; defaults to 'project'.
label	Optional label for the copied version.

Value

The new snapshot id, invisibly.

Create a project in a LibeRation workspace

Description

Create a project in a LibeRation workspace

Usage

```
nm_project_create(workspace, name, id = NULL, description = NULL)
```

Arguments

workspace	Workspace object or path.
name	Human-readable project name.
id	Optional stable project identifier.
description	Optional project description.

Value

Project metadata.

Examples

```
workspace <- nm_workspace(tempfile("liber-workspace-"))  
nm_project_create(workspace, "Example project")
```

Delete a modelling project

Description

Delete a modelling project

Usage

```
nm_project_delete(workspace, project)
```

Arguments

workspace	Workspace object or path.
-----------	---------------------------

project	Project id.
---------	-------------

Value

'TRUE', invisibly.

Delete a project snapshot

Description

Delete a project snapshot

Usage

```
nm_project_delete_snapshot(workspace, project, snapshot)
```

Arguments

workspace	Workspace object or path.
project	Project id.
snapshot	Snapshot id.

Value

'TRUE', invisibly.

List projects or stored model versions and runs

Description

List projects or stored model versions and runs

Usage

```
nm_project_list(workspace, project = NULL)
```

Arguments

workspace Workspace object or path.

project Optional project id. When supplied, model versions and runs are returned. Use the 'entry_type' and 'parent_id' columns to reconstruct the hierarchy.

Value

A data frame.

Load a project snapshot

Description

Load a project snapshot

Usage

```
nm_project_load(workspace, project, snapshot = "latest")
```

Arguments

workspace	Workspace object or path.
project	Project id.
snapshot	Snapshot id or "latest".

Value

A serializable snapshot list.

Load saved diagnostics for a model run

Description

Load saved diagnostics for a model run

Usage

```
nm_project_load_diagnostics(workspace, project, run)
```

Arguments

workspace	Workspace object or path.
project	Project id.
run	Run id.

Value

A named list of saved diagnostic objects.

Save a model version

Description

Save a model version

Usage

```
nm_project_save(  
  workspace,  
  project,  
  model = NULL,  
  data = NULL,  
  result = NULL,  
  label = NULL,  
  provenance = NULL  
)
```

Arguments

workspace	Workspace object or path.
project	Project id.
model	Optional 'nm_model' or 'NMEngine'.
data	Optional event dataset.
result	Optional legacy result. New code should store completed work with [nm_project_save_run()] so it is nested beneath a model version.
label	Model-version label. Empty labels are assigned 'Mod001', 'Mod002', and so on.

provenance

Optional named provenance fields supplied by an integrated LibeR package.
Core runtime provenance cannot be overwritten.

Value

Model-version id, invisibly.

Save diagnostics for an estimation run

Description

Save diagnostics for an estimation run

Usage

```
nm_project_save_diagnostics(workspace, project, run, diagnostics)
```

Arguments

workspace	Workspace object or path.
project	Project id.
run	Estimation-run id.
diagnostics	Named list containing saved plot data or secondary analyses for the run.

Value

The merged diagnostic list, invisibly.

Save a numbered model run

Description

A run is an estimation or simulation result owned by one immutable model version. The model and data are copied into the run record so old work remains reproducible even if the active editor changes later.

Usage

```
nm_project_save_run(  
  workspace,  
  project,  
  version,  
  result,  
  label = NULL,  
  model = NULL,  
  data = NULL,  
  queue_id = NULL,  
  queue_job_id = NULL  
)
```

Arguments

workspace	Workspace object or path.
project	Project id.
version	Parent model-version id.
result	An 'nm_fit' or simulation data frame.
label	Optional human-readable run label.

model, data

Optional run inputs; by default they are loaded from 'version'.

queue_id,
queue_job_id

Optional execution provenance used to reconcile persistent queued jobs without creating duplicate model runs.

Value

Run id, invisibly.

Build a LibeRation model from a QSP reaction system

Description

Build a LibeRation model from a QSP reaction system

Usage

```
nm_qsp_model(system, ..., EXPERIMENTAL = NULL)
```

Arguments

system	An [nm_qsp_system()].
...	Arguments passed to [nm_model()], including 'INPUT', 'PRED', parameters, likelihood, and solver controls.
EXPERIMENTAL	Explicit experimental-engine acknowledgement.

Define a sparse mechanistic QSP reaction system

Description

The stoichiometric network is compiled into the same C++/CppAD '\$DES' representation as an ordinary pharmacometric ODE. Optional algebraic constraints use the block-sparse index-1 DAE solver.

Usage

```
nm_qsp_system(  
  species,  
  stoichiometry,  
  rates,  
  dose_species = 1L,  
  observation_species = 1L,  
  algebraic = "",  
  dae_config = NULL  
)
```

Arguments

species	Unique species/state names.
stoichiometry	Species-by-reaction numeric matrix.
rates	One restricted expression per reaction. Species names in these expressions are translated to their state amounts.
dose_species, observation_species	Species name or index.
algebraic	Optional 'ALG' residual code.
dae_config	Optional [nm_dae_config()].

Define a random-effect grouping block

Description

Define a random-effect grouping block

Usage

```
nm_re_block(name, column, etas)
```

Arguments

name	Stable block label.
------	---------------------

column	Dataset column identifying the unit receiving the effect.
--------	---

etas	ETA indices belonging to the block. Correlation is permitted within a block; different blocks are independent.
------	--

Value

A serializable random-effect block declaration.

Configure nested or crossed random effects

Description

Blocks may be nested or crossed. When 'cluster' is omitted, LibeRation constructs connected components from subject and block memberships; each component becomes one conditional-likelihood unit. This is exact but a fully crossed design may form one large conditional mode. Supplying 'cluster' avoids graph discovery, provided no grouping unit spans clusters.

Usage

```
nm_re_config(..., cluster = NULL)
```

Arguments

... [nm_re_block()] declarations or one list of them.

cluster Optional dataset column defining independent top-level clusters.

Value

A serializable random-effect design.

Generate a modelling report and machine-readable manifest

Description

Produces a compact PDF report using base graphics and a JSON manifest with fit provenance and numerical summaries. The report is deterministic and does not require a browser or an external document renderer.

Usage

```
nm_report(  
  fit,  
  file,  
  sections = c("summary", "parameters", "gof", "eta", "narrative_stub"),  
  vpc = NULL,  
  title = "LibeRation model report",  
  manifest = TRUE  
)
```

Arguments

fit	An 'nm_fit'.
file	Destination PDF path.
sections	Report sections. Supported values are 'summary', 'parameters', 'gof', 'eta', 'vpc', and 'narrative_stub'.
vpc	Optional 'nm_vpc' object for the VPC section.
title	Report title.

`manifest`

Whether to write a sibling JSON manifest.

Value

An 'nm_report' object containing generated paths and metadata.

Define one block in a visual report workflow

Description

Define one block in a visual report workflow

Usage

```
nm_report_block(  
  type,  
  title = NULL,  
  source = c("user", "ai"),  
  text = "",  
  file = NULL,  
  run_ids = NULL,  
  elements = NULL,  
  options = list(),  
  id = NULL  
)
```

Arguments

type	Report-block type.
title	Display heading.
source	Narrative source: user-provided or browser-local AI. Run and comparison blocks always use saved run evidence.
text	Current editable text for narrative blocks.
file	Optional TXT, Markdown, or DOCX source for user text.
run_ids	Immutable model-run ids used by run/comparison blocks.

elements

Run evidence to include, such as summary, parameters, code, GOF, continuous/categorical/count/event VPCs, NPDE, NPC, covariance, and run information.

options

Named block options, including AI instructions, templates, and locally extracted source documents.

id

Optional stable block id.

Value

A serializable 'nm_report_block'.

Define a visual report workflow

Description

Define a visual report workflow

Usage

```
nm_report_design(  
  blocks,  
  title = "LibeRation modelling report",  
  formats = c("docx", "pdf"),  
  style = list(),  
  id = NULL  
)
```

Arguments

blocks	Ordered report blocks.
title	Report title.
formats	Output formats; DOCX and PDF are supported.
style	Named rendering options.
id	Optional stable design id.

Value

A serializable 'nm_report_design'.

Load or list project report workflows

Description

Load or list project report workflows

Usage

```
nm_report_design_load(workspace, project, id = NULL)
```

Arguments

workspace	An [nm_workspace()] or path.
project	Project id.
id	Optional report-design id. When omitted, metadata are listed.

Value

An 'nm_report_design' or metadata data frame.

Render a visual report workflow

Description

Materializes selected saved-run evidence and writes a DOCX and/or PDF report. Browser-local AI text must already have been drafted into each block; rendering never contacts an AI provider or the internet.

Usage

```
nm_report_design_render(  
  design,  
  workspace,  
  project,  
  directory = getwd(),  
  name = "liberation-report",  
  formats = design$formats,  
  manifest = TRUE  
)
```

Arguments

design	An [nm_report_design()].
workspace	An [nm_workspace()] or path.
project	Project id containing referenced runs.
directory	Output directory.
name	Output base name.
formats	DOCX and/or PDF; defaults to the design formats.

manifest

Write a JSON provenance manifest.

Value

An 'nm_report_bundle' containing generated file paths.

Save a report workflow with a project

Description

Save a report workflow with a project

Usage

```
nm_report_design_save(workspace, project, design)
```

Arguments

workspace	An [nm_workspace()] or path.
project	Project id.
design	An [nm_report_design()].

Value

The design id, invisibly.

Define a correlated residual endpoint group

Description

Declares endpoint records observed at the same subject and time as one multivariate Gaussian residual contribution. Marginal variances still come from the model's residual-error definition and 'SIGMAS'; this object defines only the cross-endpoint correlation matrix.

Usage

```
nm_residual_group(  
  dvid,  
  correlation,  
  parameter_transform = c("tanh", "identity"),  
  label = NULL  
)
```

Arguments

dvid	Unique DVID values in matrix order.
correlation	Square symmetric matrix. Diagonal values are set to one. Off-diagonal entries may be fixed numbers in '(-1, 1)' or parameter references written as 'THETA(i)' or 'SIGMA(i)'. One triangle may be left empty and is mirrored automatically.
parameter_transform	Transform for parameter references. "'tanh'" maps unconstrained parameters into '(-1, 1)'; "'identity'" uses the parameter value directly.
label	Optional human-readable group label.

Value

A serializable residual-group declaration.

Stepwise covariate modelling

Description

Candidate relationships are tested with nested re-estimation and likelihood-ratio thresholds. Continuous effects are exponential-linear, 'power' effects are normalized power functions, and categorical effects use one indicator coefficient.

Usage

```
nm_scm(  
  fit,  
  candidates,  
  direction = c("both", "forward", "backward"),  
  p_forward = 0.05,  
  p_backward = 0.01,  
  max_steps = 20L,  
  ...  
)
```

Arguments

fit	Base 'nm_fit'.
candidates	Data frame with 'parameter', 'covariate', and optional 'form', 'reference', 'category', 'scale', and 'initial' columns.
direction	Forward selection, backward elimination, or both.
p_forward, p_backward	Entry and retention significance levels.
max_steps	Maximum accepted changes in each phase.
...	Controls passed to [nm_est()].

Value

An 'nm_scm' with the final model/fit and complete selection log.

Configure a continuous-discrete stochastic differential equation

Description

Creates a compiled state-space declaration for an Ito SDE. 'drift' names one '\$ERROR' assignment per state and 'diffusion' names the square diffusion loading matrix. The C++ engine discretizes each observation interval and applies EKF, UKF, or particle inference. Milstein is available for componentwise/diagonal diffusion; Euler–Maruyama is general.

Usage

```
nm_sde_config(  
  states,  
  initial_mean,  
  initial_covariance,  
  drift,  
  diffusion,  
  observation,  
  observation_variance,  
  baseline = c("prediction", "zero"),  
  by_dvid = TRUE,  
  filter = c("ukf", "ekf", "particle"),  
  state_inputs = NULL,  
  method = c("euler", "milstein"),  
  substeps = 8L,  
  jacobian_step = 1e-05,  
  ukf_alpha = 0.5,  
  ukf_beta = 2,  
  ukf_kappa = 0,  
  particles = 256L,  
  ess_threshold = 0.5,  
  seed = 20260721L  
)
```

Arguments

states

Unique labels for latent continuous states.

<code>initial_mean</code>	Character vector naming the initial-state means.
<code>initial_covariance</code>	Square symmetric character matrix naming the initial covariance entries.
<code>drift</code>	Character vector naming the SDE drift assignments.
<code>diffusion</code>	Square character matrix naming the diffusion loading.
<code>observation</code>	For 'filter = "linear"', a character vector naming the observation-loading row 'H'. For a nonlinear filter, one assignment naming the scalar observation function.
<code>observation_variance</code>	One assignment naming the positive measurement variance.
<code>baseline</code>	Add the structural prediction 'F' to the latent-state observation mean, or use a zero baseline.
<code>by_dvid</code>	Maintain a separate latent trajectory for each DVID within subject.
<code>filter</code>	State inference algorithm: exact linear Kalman, EKF, UKF, or seeded bootstrap particle filtering.
<code>state_inputs</code>	Input symbols used for latent states by nonlinear transition and observation assignments. The default is 'STATE_<name>'.
<code>method</code>	Euler–Maruyama or Milstein propagation.
<code>substeps</code>	Fixed substeps per observation interval.

jacobian_step	Relative central-difference step used for EKF state Jacobians. Parameter derivatives are still propagated by CppAD.
ukf_alpha, ukf_beta, ukf_kappa	Scaled unscented-transform controls.
particles	Number of particles for the bootstrap filter.
ess_threshold	Resample when effective sample size falls below this proportion of 'particles'.
seed	Reproducible common-random-number seed for particle inference.

Simulate or predict a LibeRation model

Description

Simulate or predict a LibeRation model

Usage

```
nm_simulate(  
  model,  
  data,  
  theta = NULL,  
  eta = NULL,  
  sigma = NULL,  
  omega = NULL,  
  nsim = 1L,  
  random_effects = FALSE,  
  residual = FALSE,  
  sample_mixture = FALSE,  
  censor = FALSE,  
  seed = NULL,  
  n_cores = 1L  
)
```

Arguments

model	An 'nm_model' or compiled 'NMEngine'.
data	NONMEM-style event data.
theta	Population fixed effects; defaults to the model table.
eta	Subject ETA matrix ordered by first subject appearance.
sigma	Residual parameters.

omega	Random-effect covariance parameters.
nsim	Number of simulation replicates.
random_effects	Sample subject/occasion ETAs from OMEGA when 'eta' is not supplied.
residual	Sample observations from the configured residual model.
sample_mixture	Sample finite-mixture membership per subject.
censor	Mark simulated values below LLOQ as BLQ/CENS and replace their reported DV by the applicable quantification limit.
seed	Optional reproducible RNG seed.
n_cores	Number of parallel simulation workers. Replicates are distributed across persistent PSOCK workers on Windows, Linux, and macOS.

Value

Input records augmented with 'IPRED', compartment amounts, and—when requested—simulated 'DV', ETAs, mixture membership, and replicate number.

Examples

```
model <- nm_model(
  INPUT = c("ID", "TIME", "EVID", "AMT", "CMT", "DV", "MDV"),
  ADVAN = 1,
  PRED = "CL=THETA(1); V=THETA(2); S1=V",
  ERROR = "Y=F",
  THETAS = data.frame(THETA = 1:2, Value = c(2, 20))
)
events <- data.frame(
```

```
ID = 1, TIME = c(0, 1, 2), EVID = c(1, 0, 0),  
  AMT = c(100, 0, 0), CMT = 1, DV = NA_real_, MDV = c(1, 0, 0)  
)  
nm_simulate(model, events)
```

Generate a restricted cubic spline expression

Description

Generate a restricted cubic spline expression

Usage

```
nm_spline(x = "TIME", knots, coefficients, intercept = "0")
```

Arguments

x	Predictor symbol, usually 'TIME' or a covariate.
knots	At least three strictly increasing knots.
coefficients	Coefficients for the linear term followed by the 'length(knots) - 2' nonlinear basis terms.
intercept	Optional intercept expression.

Value

Editable restricted-cubic-spline expression using only tape-safe arithmetic and 'pmax()'.

Catalogue of advanced structural model templates

Description

Catalogue of advanced structural model templates

Usage

```
nm_structural_templates()
```

Value

A data frame describing templates accepted by [nm_model_template()].

Report implemented feature support

Description

Report implemented feature support

Usage

```
nm_support_matrix()
```

Configure an experimental switching nonlinear state-space model

Description

A discrete latent regime is propagated jointly with the continuous state by the compiled particle filter. A deterministic stratified regime proposal and differentiable importance weights preserve transition-parameter gradients, while each regime has its own nonlinear transition (or SDE drift/diffusion) and observation model. Regime filtering and genealogical smoothing are returned by `[nm_kalman_decode()]`.

Usage

```
nm_switching_state_space_config(  
  regimes,  
  initial_regime,  
  regime_transition,  
  states,  
  initial_mean,  
  initial_covariance,  
  transition,  
  process_covariance,  
  observation,  
  observation_variance,  
  initial_scale = c("probability", "log"),  
  transition_scale = c("probability", "log"),  
  baseline = c("prediction", "zero"),  
  by_dvid = TRUE,  
  state_inputs = NULL,  
  particles = 512L,  
  ess_threshold = 0.5,  
  seed = 20260721L,  
  dynamics = c("discrete", "sde"),  
  sde_method = c("euler", "milstein"),  
  sde_substeps = 8L,  
  jacobian_step = 1e-05  
)
```

Arguments

regimes	Unique regime labels.
initial_regime	Assignment names for initial regime weights.
regime_transition	Square matrix of assignment names for regime transition weights.
states	Unique labels for latent continuous states.
initial_mean	Character vector naming the initial-state means.
initial_covariance	Square symmetric character matrix naming the initial covariance entries.
transition	List with one next-state/drift assignment vector per regime.
process_covariance	List with one covariance/diffusion matrix per regime.
observation, observation_variance	One assignment per regime.
initial_scale, transition_scale	Probability or log weights.
baseline	Add the structural prediction 'F' to the latent-state observation mean, or use a zero baseline.
by_dvid	Maintain a separate latent trajectory for each DVID within subject.

state_inputs	Input symbols used for latent states by nonlinear transition and observation assignments. The default is 'STATE_<name>'.
particles	Number of particles for the bootstrap filter.
ess_threshold	Resample when effective sample size falls below this proportion of 'particles'.
seed	Reproducible common-random-number seed for particle inference.
dynamics	Discrete transition assignments or continuous-discrete SDE drift/diffusion assignments. Prefer [nm_sde_config()] for SDE models.
sde_method	Fixed-step Euler–Maruyama or diagonal Milstein propagation for SDE dynamics.
sde_substeps	Number of fixed SDE steps per observation interval.
jacobian_step	Relative central-difference step used for EKF state Jacobians. Parameter derivatives are still propagated by CppAD.

Require numerical agreement before publishing a benchmark

Description

Require numerical agreement before publishing a benchmark

Usage

```
nm_validation_gate(  
  reference,  
  candidate,  
  reference_predictions = NULL,  
  candidate_predictions = NULL,  
  tolerances = list(objective = list(absolute = 0.001, relative = 1e-04),  
  theta =  
    list(absolute = 1e-05, relative = 0.02), omega = list(absolute = 1e-  
05, relative =  
  0.1), sigma = list(absolute = 1e-05, relative = 0.1), eta =  
list(absolute = 1e-04,  
  relative = 0.1), predictions = list(absolute = 1e-06, relative =  
0.02)),  
  required = c("objective", "theta", "omega", "sigma", "eta",  
"predictions")  
)
```

Arguments

reference, candidate	Reference-engine and LibeRation fit summaries or 'nm_fit' objects.
reference_predictions, candidate_predictions	Optional prediction vectors, avoiding a fresh 'predict()' call.
tolerances	Named per-component absolute/relative tolerances.

required

Components that must be present and pass.

Value

An 'nm_validation_gate' report.

Visual predictive check summaries

Description

Visual predictive check summaries

Usage

```
nm_vpc(  
  fit,  
  nsim = 200L,  
  breaks = NULL,  
  probs = c(0.05, 0.5, 0.95),  
  level = 0.9,  
  seed = 20260713L,  
  pc_correct = FALSE,  
  stratify = NULL  
)
```

Arguments

fit	An 'nm_fit'.
nsim	Number of stochastic replicates.
breaks	Optional time-bin breaks. The default uses quantile bins.
probs	Observation quantiles.
level	Simulation interval for each quantile.
seed	RNG seed.

pc_correct

Apply the legacy prediction correction 'DV * PRED / IPRED' to observed and simulated values before bin summaries.

stratify

Optional dataset column used to create additional stratum-specific VPC summaries. The unstratified VPC is always retained.

Value

An 'nm_vpc' list containing observed and simulated summaries.

Categorical visual predictive check

Description

First-class Bernoulli, categorical, ordinal, and Markov outcomes use their declared probability vectors and can contain any number of categories. A legacy binary user likelihood remains supported when 'F' is the probability of the non-reference category.

Usage

```
nm_vpc_categorical(  
  fit,  
  outcome = "DV",  
  nsim = 200L,  
  breaks = NULL,  
  level = 0.9,  
  seed = 20260713L  
)
```

Arguments

fit	An 'nm_fit'.
outcome	Binary observed outcome column.
nsim	Number of simulations.
breaks	Optional time bins.
level	Simulation interval.
seed	RNG seed.

Value

An 'nm_vpc_categorical'.

Competing-risk visual predictive check

Description

Uses Aalen-Johansen cumulative incidence curves for every declared cause and compares them with simulation intervals from a first-class 'competing_risks' outcome.

Usage

```
nm_vpc_competing(  
  fit,  
  event = "DV",  
  dvid = NULL,  
  nsim = 200L,  
  level = 0.9,  
  seed = 20260713L  
)
```

Arguments

fit	An 'nm_fit'.
event	Cause-code column ('0' denotes no event/censoring).
dvid	Optional joint-endpoint 'DVID'.
nsim	Number of simulations.
level	Simulation interval.
seed	RNG seed.

Value

An 'nm_vpc_competing'.

Count visual predictive check

Description

Summarizes the mean, variance, zero fraction, median, and upper count quantile in time bins for first-class Poisson, negative-binomial, binomial, ZIP, and hurdle models.

Usage

```
nm_vpc_count(  
  fit,  
  outcome = "DV",  
  dvid = NULL,  
  nsim = 200L,  
  breaks = NULL,  
  level = 0.9,  
  seed = 20260713L  
)
```

Arguments

fit	An 'nm_fit'.
outcome	Count column, normally 'DV'.
dvid	Optional endpoint 'DVID' in a joint model.
nsim	Number of simulations.
breaks	Optional time bins.
level	Simulation interval.

seed

RNG seed.

Value

An 'nm_vpc_count'.

Recurrent-event visual predictive check

Description

Compares the observed mean cumulative event function with predictive intervals from a first-class 'recurrent_event' outcome.

Usage

```
nm_vpc_recurrent(  
  fit,  
  event = "DV",  
  dvid = NULL,  
  nsim = 200L,  
  level = 0.9,  
  seed = 20260713L  
)
```

Arguments

fit	An 'nm_fit'.
event	Event-indicator column.
dvid	Optional joint-endpoint 'DVID'.
nsim	Number of simulations.
level	Simulation interval.
seed	RNG seed.

Value

An 'nm_vpc_recurrent'.

Time-to-event visual predictive check

Description

'F'/IPRED' is interpreted as a non-negative instantaneous hazard on each subject's observation grid. The first event is simulated from the integrated piecewise-constant hazard and Kaplan-Meier curves are summarized across simulations.

Usage

```
nm_vpc_tte(fit, event = "DV", nsim = 200L, level = 0.9, seed = 20260713L)
```

Arguments

fit	An 'nm_fit'.
event	Binary event-indicator column.
nsim	Number of simulations.
level	Simulation interval.
seed	RNG seed.

Value

An 'nm_vpc_tte'.

Create or open a modelling workspace

Description

A workspace stores serializable model versions, their runs, and saved diagnostics. Compiled external pointers are deliberately excluded and rebuilt when a model version or run is opened.

Usage

```
nm_workspace(  
  path = file.path(tools::R_user_dir("LibeRation", "data"), "workspace")  
)
```

Arguments

path	Workspace directory.
------	----------------------

Value

An 'nm_workspace' object.

Examples

```
workspace <- nm_workspace(tempfile("liber-workspace-"))  
workspace
```

Create a consistent workspace backup

Description

Create a consistent workspace backup

Usage

```
nm_workspace_backup(workspace, path = NULL)
```

Arguments

workspace	Workspace object or path.
-----------	---------------------------

path	Destination '.tar.gz'; defaults to the workspace backup folder.
------	---

Value

Normalized backup path, invisibly.

Remove unreferenced content-addressed workspace objects

Description

Remove unreferenced content-addressed workspace objects

Usage

```
nm_workspace_gc(workspace, dry_run = TRUE)
```

Arguments

workspace Workspace object or path.

dry_run Report candidates without deleting them. This defaults to 'TRUE' deliberately.

Value

A data frame of unreferenced objects.

Migrate an existing workspace to the current schema

Description

Manifest upgrades are lossless. With 'convert_snapshots = TRUE', legacy snapshot payloads are rewritten to content-addressed v2 storage after a backup has been created.

Usage

```
nm_workspace_migrate(workspace, convert_snapshots = FALSE, backup = TRUE)
```

Arguments

workspace	Workspace object or path.
convert_snapshots	Convert legacy embedded snapshots.
backup	Create a complete backup before conversion.

Value

Migration counts, invisibly.

Verify a LibeRation workspace

Description

Checks manifests, snapshot contracts, content-addressed objects, and hashes without changing the workspace.

Usage

```
nm_workspace_verify(workspace)
```

Arguments

workspace	Workspace object or path.
-----------	---------------------------

Value

A 'liber_workspace_verification' report.

Compiled LibeRation numerical engine

Description

The R6 object owns a disposable C++ pointer. Its 'model' field remains fully serializable and is the object sent to workers.

Public fields

model

Serializable validated [nm_model()] definition.

pointer

External pointer to the compiled C++ numerical engine.

Methods

Public methods

- `NMEngine$new()`
- `NMEngine$simulate()`
- `NMEngine$hmm_filter()`
- `NMEngine$kalman_filter()`
- `NMEngine$prediction_tape()`
- `NMEngine$prediction_derivatives()`
- `NMEngine$objective_tape()`
- `NMEngine$objective()`
- `NMEngine$print()`
- `NMEngine$clone()`

`NMEngine$new()`

Compile a serializable model into a C++ engine.

Usage

```
NMEngine$new(model)
```

Arguments

`model`

A validated `[nm_model()]`.

Returns

A new 'NMEngine' object.

NMEngine\$simulate()

Run deterministic event-table prediction in C++.

Usage

```
NMEngine$simulate(  
  data,  
  theta = self$model$THETAS$Value,  
  eta = NULL,  
  sigma = self$model$SIGMAS$Value  
)
```

Arguments

`data`

NONMEM-style event data.

`theta`

Population fixed effects.

`eta`

Subject-by-effect random-effect matrix.

`sigma`

Residual parameters.

Returns

Event data augmented with 'IPRED' and compartment amounts.

NMEngine\$hmm_filter()

Run the scaled hidden Markov forward filter.

Usage

```
NMEngine$hmm_filter(  
  data,  
  theta = self$model$THETAS$Value,  
  eta = NULL,  
  sigma = self$model$SIGMAS$Value  
)
```

Arguments

data

NONMEM-style observation data.

theta

Population fixed effects.

eta

Subject-by-effect random-effect matrix.

sigma

Residual parameters used by emission expressions.

Returns

Filtered hidden-state probabilities and likelihood details.

NMEngine\$kalman_filter()

Run the linear Gaussian Kalman filter and RTS smoother.

Usage

```
NMEngine$kalman_filter(  
  data,  
  theta = self$model$THETAS$Value,  
  eta = NULL,  
  sigma = self$model$SIGMAS$Value  
)
```

Arguments

data

NONMEM-style observation data.

theta

Population fixed effects.

eta

Subject-by-effect random-effect matrix.

sigma

Parameters used by state-space component expressions.

Returns

Filtered and smoothed state summaries and likelihood details.

NMEngine\$prediction_tape()

Record the complete prediction calculation on a persistent AD tape.

Usage

```
NMEngine$prediction_tape(  
  data,  
  theta = self$model$THETAS$Value,  
  eta = NULL,  
  sigma = self$model$SIGMAS$Value  
)
```

Arguments

data

NONMEM-style event data.

theta

Population fixed effects.

eta

Subject-by-effect random-effect matrix.

sigma

Residual parameters.

Returns

An internal 'nm_prediction_tape' object.

NMEngine\$prediction_derivatives()

Evaluate predictions and their exact automatic derivatives.

Usage

```
NMEngine$prediction_derivatives(  
  data,  
  theta = self$model$THETAS$Value,  
  eta = NULL,  
  sigma = self$model$SIGMAS$Value,  
  jacobian = TRUE  
)
```

Arguments

data

NONMEM-style event data.

theta

Population fixed effects.

eta

Subject-by-effect random-effect matrix.

`sigma`

Residual parameters.

`jacobian`

Whether to return the full Jacobian.

Returns

Prediction values, optional Jacobian, and AD domain names.

NMEngine\$objective_tape()

Record the joint population objective on a persistent AD tape.

Usage

```
NMEngine$objective_tape(  
  data,  
  theta = self$model$THETAS$Value,  
  eta = NULL,  
  sigma = self$model$SIGMAS$Value,  
  omega = self$model$OMEGAS$Value,  
  interaction = TRUE  
)
```

Arguments

`data`

NONMEM-style event data with observations.

`theta`

Population fixed effects.

`eta`

Subject-by-effect random-effect matrix.

`sigma`

Residual parameters.

omega

Random-effect covariance parameters.

interaction

Include ETA-residual interaction.

Returns

An internal 'nm_objective_tape' object.

NMEngine\$objective()

Evaluate the exact joint objective and requested derivatives.

Usage

```
NMEngine$objective(  
  data,  
  theta = self$model$THETAS$Value,  
  eta = NULL,  
  sigma = self$model$SIGMAS$Value,  
  omega = self$model$OMEGAS$Value,  
  gradient = TRUE,  
  hessian = FALSE,  
  interaction = TRUE  
)
```

Arguments

data

NONMEM-style event data with observations.

theta

Population fixed effects.

eta

Subject-by-effect random-effect matrix.

`sigma`

Residual parameters.

`omega`

Random-effect covariance parameters.

`gradient`

Whether to return the exact gradient.

`hessian`

Whether to return the exact Hessian.

`interaction`

Include ETA-residual interaction.

Returns

Objective value, requested derivatives, and AD domain names.

NMEngine\$print()

Print a concise engine summary.

Usage

```
NMEngine$print(...)
```

Arguments

...

Unused.

Returns

The engine, invisibly.

NMEngine\$clone()

The objects of this class are cloneable with this method.

Usage

```
NMEngine$clone(deep = FALSE)
```

Arguments

deep

Whether to make a deep clone.

Predictions from a fitted LibeRation model

Description

Predictions from a fitted LibeRation model

Usage

```
## S3 method for class 'nm_fit'  
predict(object, newdata = NULL, type = c("individual", "population"), ...)
```

Arguments

object
An 'nm_fit'.

newdata
Optional event dataset. Individual predictions currently require the original subject and occasion layout.

type
Individual or population predictions.

...
Reserved.

Value

Event data augmented by fitted predictions and state amounts.

Render a LibeR workbench in Shiny

Description

Render a LibeR workbench in Shiny

Usage

```
renderLiberWorkbench(expr, env = parent.frame(), quoted = FALSE)
```

Arguments

expr	Expression returning [liber_workbench()].
------	---

env	Evaluation environment.
-----	-------------------------

quoted	Whether 'expr' is quoted.
--------	---------------------------

Residual diagnostics for a fitted model

Description

Residual diagnostics for a fitted model

Usage

```
## S3 method for class 'nm_fit'  
residuals(object, type = c("IWRES", "CWRES", "WRES", "IRES", "RES"), ...)
```

Arguments

object	An 'nm_fit'.
type	Residual column to return.
...	Reserved.

Value

A numeric residual vector aligned to event records.